DD: SUB-WF-SUSPEND-NP-01-FLOW-WIK — KE WIK Non-Payment Suspension BPMN

Developer implementation guide: `DD_SUB-WF-SUSPEND-NP-01-FLOW-WIK-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow `DD_SUB-WF-SUSPEND-NP-01-FLOW-WIK-v1.0.md`.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0 Satellite of **DD_SUB-WF-SUSPEND-NP-01-Subscription_Non_Payment_Suspension** (parent). Defines the KE WIK reference BPMN for non-payment suspension. BPMN process key: `sub-suspend-np-wik`. Composes 12 workers from the framework's WORKERS catalog, 1 message catch, 1 Drools package. No user task (system-driven only). No pay-first gate (suspension is the consequence of non-payment, not a billable event).

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-SUSPEND-NP-01-FLOW-WIK-v1.0.md`
Version	`v1.0`
DD type	`operator BPMN flow`
BPMN/process keys	`sub-suspend-np-wik`
Drools packages	`rules.subscription.suspend-np.wik`

4. Runtime Contracts To Implement

4.1 APIs

- `POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/suspend-np`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;

- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `bil01-emit-intent`
- `drools-decide`
- `ful-call-restrict`
- `fulfillment-completed`
- `fulfillment-failed`
- `kafka-emit`
- `manual-recovery-decision`
- `notification-send`
- `sub-lm-cancel-in-flight-operation`
- `sub-lm-commit-state`
- `sub-lm-compute-cycle-window`
- `sub-lm-put-active-restrictions`
- `sub-lm-put-master-fields`
- `sub-lm-put-pending-status`
- `sub-lm-revert-to-prior`
- `sub-suspend-np-wik`
- `subscription-operation-finalize`
- `subscription-workflow-rules`
- `validate-preconditions`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `SubscriptionSuspendNPRejected`
- `SubscriptionSuspendedForNonPayment`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.

4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0

Satellite of **DD_SUB-WF-SUSPEND-NP-01-Subscription_Non_Payment_Suspension** (parent). Defines the KE WIK reference BPMN for non-payment suspension. BPMN process key: `sub-suspend-np-wik`. Composes 12 workers from the framework's WORKERS catalog, 1 message catch, 1 Drools package. No user task (system-driven only). No pay-first gate (suspension is the consequence of non-payment, not a billable event).

1. What this flow is

KE WIK's BPMN composes the framework workers to enact a non-payment suspension. Trigger endpoint accepts the request from BIL-04 (BILLING_INTERNAL only), persists `subscription_operation` row, starts process instance of `sub-suspend-np-wik`. The BPMN orchestrates: Drools validation → cancel any in-flight operation → pending state flip → compute cycle window → billing intent (analytics-only, no charge) → FUL-04 SUSPEND_SERVICE → clear all active restrictions (per SUB-LM-01 R-RES-6) → master fields commit → pause-history row insert (via master-fields with `originIntent=SUSPEND_NP`, `systemManaged=true`) → commit state SUSPENDED → emit `SubscriptionSuspendedForNonPayment` → notify customer → finalize.

Workers composed (12)

#	Worker topic	Used for
1	`drools-decide`	Validation: role=BILLING_INTERNAL, subscription ACTIVE, dunning context completeness, debt-tier computation
2	`sub-lm-cancel-in-flight-operation`	Cancel any other in-flight state-affecting operation on this subscription (PAUSE, UPGRADE, etc.) before proceeding
3	`sub-lm-put-pending-status`	Flip subscription to `PENDING_SUSPEND_NP`; record `current_transition_type=SUSPEND_NP`, `current_transition_reason_code=<dunningReasonCode>`

#	Worker topic	Used for
4	`sub-lm-compute-cycle-window`	Compute cycle window for billing intent context
5	`bil01-emit-intent`	`SUSPEND_NP_INTENT` — billing records suspension, freezes accrual per policy, no charge
6	`ful-call-restrict`	FUL-04 with `intent=SUSPEND_SERVICE`, `reasonCategory=NON_PAYMENT`, `dunningEscalationLevel`
7	`sub-lm-put-active-restrictions`	`CLEAR_ALL` — wipe `active_restrictions[]` per R-RES-6 (suspension already revokes all service)
8	`sub-lm-put-master-fields`	Write `last_status_changed_at`; insert pause-history row with `originIntent=SUSPEND_NP`, `systemManaged=true`, dunning context
9	`sub-lm-commit-state`	Flip subscription to `SUSPENDED`
10	`kafka-emit`	Emit `SubscriptionSuspendedForNonPayment` via outbox
11	`notification-send`	NOT-01 customer SMS (reliability-isolated; only if s `ubscription_suspend_np_config.customer _notification_enabled=true` for the operator)
12	`subscription-operation-finalize`	Set `subscription_operation.final_state= COMPLETED`

Message catches

Type	Name	Used when
Message catch	`fulfillment-completed` / `fulfillment-failed` (correlated on `fulfillmentCorrelationId`)	After `ful-call-restrict`; BPMN waits for FUL-04

Drools package

`rules.subscription.suspend-np.wik` — minimal (system-driven flow with limited variability):

Decision	Output	Used at
`validate-preconditions`	`eligible: bool`, `eligibilityReasonCode?`, `debtAmountTier: str`	Beginning — subscription ACTIVE, role=BILLING_INTERNAL, dunning context fields present, debt-tier classification

2. BPMN structure

Process key: `sub-suspend-np-wik`. Deployed as `sub-suspend-np-wik.bpmn` in the Camunda engine.

2.1 Outline

```
START (event)
  ■ Carries process variables from trigger endpoint:
  ■   operationId, subscriptionId, customerId, operatorCode, packageRef, homepassId,
  ■   dunningReasonCode, dunningCycleReference, dunningEscalationLevel,
  ■   outstandingDebtAmount, outstandingDebtCurrency, notes,
  ■   initiatingActorUserId, initiatingActorRole, initiatingWorkflowKind,
  ■   initiatingWorkflowRef, idempotencyKey
  ■
  ▼
[STATE: VALIDATING]
Service Task: drools-decide
  inputs: kjarPackage="rules.subscription.suspend-np.wik",
          factsJson={subscription state, role, dunning context, operator config},
          requestedDecisions=["validate-preconditions"]
  outputs: eligible, eligibilityReasonCode, debtAmountTier
  ■
  ▼ XOR: eligible?
  ■ NO → REJECT subflow (emit SubscriptionSuspendNPRejected + finalize FAILED)
  ■ YES ↓
  ■
  ▼
[STATE: PENDING_STATE_FLIP]
```

```
XOR gateway: subscription in PENDING_* (in-flight operation)?
  ■ YES ↓
  ■   Service Task: sub-lm-cancel-in-flight-operation
  ■     inputs: subscriptionId, cancelReason="SUPERSEDED_BY_SUSPEND_NP"
  ■   Wait: up to 60s for prior op to acknowledge cancellation (Camunda boundary timer)
  ■ NO ↓
  ■
Service Task: sub-lm-put-pending-status
  inputs: subscriptionId, toStatus="PENDING_SUSPEND_NP",
          currentTransitionType="SUSPEND_NP",
          currentTransitionReasonCode=<dunningReasonCode>
  outputs: priorStatusSnapshot, lmEventId
  ■
  ▼
Service Task: sub-lm-compute-cycle-window
  inputs: subscriptionId, referenceDate=NOW
  outputs: cycleWindowJson
  ■
  ▼
[STATE: BILLING_CALL]
Service Task: bil01-emit-intent
  inputs: subscriptionId,
          intentKind="SUSPEND_NP_INTENT",
          intentContext={dunningReasonCode, dunningCycleReference, dunningEscalationLevel,
                         outstandingDebtAmount, outstandingDebtCurrency, debtAmountTier},
          cycleWindow
  outputs: intentDecisionId, intentDecision (lineItems usually empty — analytics only)
  ■
  ▼
[STATE: FULFILLMENT_CALL]
Service Task: ful-call-restrict
  inputs: subscriptionId,
          intent="SUSPEND_SERVICE",
          reasonCategory="NON_PAYMENT",
          dunningEscalationLevel,
          fulfillmentCorrelationId=<operationId>:suspend-np
  outputs: restrictionFulfillmentId, restrictionTerminalState
  ■
  ▼ [STATE: AWAITING_FULFILLMENT_RESPONSE]
Message Catch: fulfillment-completed (correlated on fulfillmentCorrelationId)
  OR
Message Catch: fulfillment-failed
  ■ FAILED → REVERT subflow (sub-lm-revert-to-prior to ACTIVE + bil01-revert-pending-intent + finalize FAILED + emit Reje
  ■ COMPLETED ↓
  ■
Service Task: sub-lm-put-active-restrictions
  inputs: subscriptionId, restrictionsOperationKind="CLEAR_ALL"
  outputs: lmEventId, clearedRestrictions (captured for event payload)
  ■
  ▼
[STATE: COMMITTING_FINAL_STATE]
Service Task: sub-lm-put-master-fields
  inputs: subscriptionId,
          fieldsJson={last_status_changed_at=NOW},
          pauseHistoryRow={originIntent="SUSPEND_NP", systemManaged=true,
                           dunningReasonCode, dunningCycleReference, dunningEscalationLevel,
                           outstandingDebtAmount, outstandingDebtCurrency,
                           suspendedAt=NOW, notes}
  outputs: lmEventId, pauseHistoryRowId
  ■
  ▼
Service Task: sub-lm-commit-state
  inputs: subscriptionId, toStatus="SUSPENDED", expectedFromStatus="PENDING_SUSPEND_NP"
  outputs: committedAt
  ■
  ▼
[STATE: EMITTING_EVENT]
Service Task: kafka-emit
  inputs: topic="sophix.subscription.subscription.suspendednonpayment",
          key=<subscriptionId>,
          payloadJson=<full SubscriptionSuspendedForNonPayment payload per parent §6>
  ■
  ▼
Service Task: notification-send (reliability-isolated, conditional on operator config)
  inputs: recipientKind="customer", recipientRef=<customerId>,
          notificationEventKind="SUBSCRIPTION_SUSPENDED_NON_PAYMENT",
          eventContext={dunningReasonCode, outstandingDebtAmount, outstandingDebtCurrency,
                        debtAmountTier, suspendedAt}
  ■ failure here does NOT fail the BPMN
  ▼
[STATE: COMPLETED]
Service Task: subscription-operation-finalize
```

```
  inputs: operationId, finalState="COMPLETED"
  ■
END (event)
```

2.2 BPMN XML excerpt — clear restrictions + master fields + commit

```
<bpmn:serviceTask id="Task_clear_restrictions" name="Clear all active restrictions"
                  camunda:type="external" camunda:topic="sub-lm-put-active-restrictions">
  <bpmn:extensionElements>
    <camunda:inputOutput>
      <camunda:inputParameter name="restrictionsOperationKind">CLEAR_ALL</camunda:inputParameter>
    </camunda:inputOutput>
  </bpmn:extensionElements>
</bpmn:serviceTask>

<bpmn:serviceTask id="Task_put_master_fields" name="Write master fields + pause history"
                  camunda:type="external" camunda:topic="sub-lm-put-master-fields">
  <bpmn:extensionElements>
    <camunda:inputOutput>
      <camunda:inputParameter name="pauseHistoryRow">
        {
          "originIntent": "SUSPEND_NP",
          "systemManaged": true,
          "dunningReasonCode": "${dunningReasonCode}",
          "dunningCycleReference": "${dunningCycleReference}",
          "dunningEscalationLevel": ${dunningEscalationLevel},
          "outstandingDebtAmount": ${outstandingDebtAmount},
          "outstandingDebtCurrency": "${outstandingDebtCurrency}",
          "suspendedAt": "${now}",
          "notes": "${notes}"
        }
      </camunda:inputParameter>
    </camunda:inputOutput>
  </bpmn:extensionElements>
</bpmn:serviceTask>

<bpmn:serviceTask id="Task_commit_state" name="Commit SUSPENDED"
                  camunda:type="external" camunda:topic="sub-lm-commit-state">
  <bpmn:extensionElements>
    <camunda:inputOutput>
      <camunda:inputParameter name="toStatus">SUSPENDED</camunda:inputParameter>
      <camunda:inputParameter name="expectedFromStatus">PENDING_SUSPEND_NP</camunda:inputParameter>
    </camunda:inputOutput>
  </bpmn:extensionElements>
</bpmn:serviceTask>
```

2.3 Compensation paths

If this step fails	Compensation BPMN runs
`drools-decide` validation	No mutations yet; emit `SubscriptionSuspendNPRejected`; finalize FAILED
`sub-lm-cancel-in-flight-operation` timeout	Mark operation FAILED with `IN_FLIGHT_CANCEL_TIMEOUT`; manual NOC review
`sub-lm-put-pending-status`	Atomic — failure leaves subscription ACTIVE; emit Rejected; finalize FAILED
`bil01-emit-intent`	`sub-lm-revert-to-prior` to ACTIVE; emit Rejected; finalize FAILED
`ful-call-restrict` returns FAILED	`sub-lm-revert-to-prior` to ACTIVE; emit Rejected; finalize FAILED. FUL-04 internally handles network-side compensation.
`sub-lm-put-active-restrictions` (CLEAR_ALL)	Network suspension already applied; manual recovery via `manual-recovery-decision` user task (restrictions array out of sync)
`sub-lm-put-master-fields` / pause-history insert	Same as above — manual recovery
`sub-lm-commit-state`	Manual recovery via `manual-recovery-decision` user task; finalize FAILED
`kafka-emit`	Retried by outbox dispatcher; BPMN proceeds
`notification-send`	Logged but NOT compensated (reliability-isolated)

2.4 Configuration row (KE WIK)

In `subscription_operation_config` (framework parent):

operator_code	operation_kind	default_bpmn_process_key	operation_timeout_seconds	feature_flags
`WIK`	`SUSPEND_NP`	`sub-suspend-np-wik`	120	`{}`

In `subscription_suspend_np_config` (parent §3.1):

operator_code	customer_notification_enabled	debt_amount_warning_threshold	debt_amount_warning_currency
`WIK`	true	5000.00	`KES`

3. Drools package `rules.subscription.suspend-np.wik`

KE WIK rules deployed as part of `subscription-workflow-rules` KJAR.

3.1 `validate-preconditions`

```
package rules.subscription.suspend-np.wik

import com.sophix.subwf.facts.SuspendNpRequestFact
import com.sophix.subwf.facts.SubscriptionStateFact
import com.sophix.subwf.facts.OperatorConfigFact

rule "Eligibility: subscription must be ACTIVE"
when
  $req: SuspendNpRequestFact()
  $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, statusCode != "ACTIVE")
then
  modify($req) { setEligible(false); setEligibilityReasonCode("INVALID_STATE_FOR_SUSPEND_NP") }
end

rule "Eligibility: role must be BILLING_INTERNAL"
when
  $req: SuspendNpRequestFact(eligible == null, initiatingActorRole != "BILLING_INTERNAL")
then
  modify($req) { setEligible(false); setEligibilityReasonCode("UNAUTHORIZED_TRIGGER") }
end

rule "Eligibility: dunning context fields required"
when
  $req: SuspendNpRequestFact(eligible == null)
  eval($req.getDunningReasonCode() == null || $req.getDunningCycleReference() == null
       || $req.getOutstandingDebtAmount() == null || $req.getOutstandingDebtCurrency() == null)
then
  modify($req) { setEligible(false); setEligibilityReasonCode("MISSING_DUNNING_CONTEXT") }
end

rule "Debt tier: HIGH"
when
  $req: SuspendNpRequestFact(eligible == null || eligible == true)
  $cfg: OperatorConfigFact(debtAmountWarningThreshold != null)
  eval($req.getOutstandingDebtAmount() != null
       && $req.getOutstandingDebtCurrency().equals($cfg.getDebtAmountWarningCurrency())
       && $req.getOutstandingDebtAmount().compareTo($cfg.getDebtAmountWarningThreshold()) >= 0)
then
  modify($req) { setDebtAmountTier("HIGH") }
end

rule "Debt tier: STANDARD (default)"
salience -10
when
  $req: SuspendNpRequestFact(debtAmountTier == null)
then
  modify($req) { setDebtAmountTier("STANDARD") }
end

rule "Eligibility: default-pass"
salience -100
when
  $req: SuspendNpRequestFact(eligible == null)
then
  modify($req) { setEligible(true) }
end
```

4. Worked example — Kamau hits L3 after L2 voice-bar failed to recover payment

Context: Kamau was placed on `OUTGOING_VOICE_BARRED` at BIL-04 dunning L2 on 2026-05-21 (see SUB-WF-RESTRICT-01-FLOW-WIK worked example). 14 days pass without payment of INV_01HZ. BIL-04 advances to L3

on 2026-06-15. Outstanding debt: KES 3,500 (one month). KE WIK threshold is KES 5,000 → debt tier STANDARD.

02:00:00 — Trigger endpoint:

```
POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/suspend-np
Idempotency-Key: suspend-np-sub_01HZ_KAMAU-dunning-wave-2026-06-15-L3

{ "operatorCode": "WIK",
  "dunningReasonCode": "DUNNING_ESCALATION_T3",
  "dunningCycleReference": "bil04-dunning-wave-2026-06-15-L3",
  "dunningEscalationLevel": 3,
  "outstandingDebtAmount": 3500.00, "outstandingDebtCurrency": "KES",
  "notes": "Dunning L3 reached after L2 voice-bar did not produce payment within 14 days",
  "initiatingActorUserId": "bil04-svc-account",
  "initiatingActorRole": "BILLING_INTERNAL",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "bil04-dunning-wave-2026-06-15-L3" }
→ 202 operationId=subop_01HZ_KAMAU_SUSPNP, currentState=VALIDATING
```

`subscription_operation` row inserted. Camunda starts `pi_01HZ_KAMAU_SUSPNP` of `sub-suspend-np-wik`.

02:00:00.5 — VALIDATING: `drools-decide` invokes `rules.subscription.suspend-np.wik`. Facts: Kamau's subscription ACTIVE, role BILLING_INTERNAL, all dunning fields present, debt KES 3,500 < KES 5,000 threshold. Returns `eligible=true, debtAmountTier=STANDARD`.

02:00:01 — PENDING_STATE_FLIP: No in-flight operations on Kamau. `sub-lm-put-pending-status` flips subscription to PENDING_SUSPEND_NP; sets `current_transition_type=SUSPEND_NP`, `current_transition_reason_code=DUNNING_ESCALATION_T3`.

02:00:01.5: `sub-lm-compute-cycle-window` returns `{cycleStart:"2026-06-01", cycleEnd:"2026-06-30"}`.

02:00:02 — BILLING_CALL: `bil01-emit-intent` calls BIL-01 with `intentKind=SUSPEND_NP_INTENT`, context with full dunning fields + debt-tier. BIL-01 records suspension state, freezes June cycle accrual, emits analytics BillableEvent `BIL_EVT_NP_SUSPEND_T3` (no charge). Returns `lineItems=[], totals.net=0`.

02:00:03 — FULFILLMENT_CALL: `ful-call-restrict` calls FUL-04 with `intent=SUSPEND_SERVICE, reasonCategory=NON_PAYMENT, dunningEscalationLevel=3, fulfillmentCorrelationId=subop_01HZ_KAMAU_SUSPNP:suspend-np`. FUL-04 dispatches to VoipSipProvisioner (`suspendService` → disable Kamau's voice profile in VoipSwitch entirely) and GponResProvisioner (`suspendService` → AAA disable for Kamau's ONT, rate-zero the OLT port). Returns `terminalState=FULLY_FULFILLED` after 2.3 seconds.

02:00:05.3 — AWAITING_FULFILLMENT_RESPONSE: Camunda message catch `fulfillment-completed` fires.

02:00:05.5: `sub-lm-put-active-restrictions` with `restrictionsOperationKind=CLEAR_ALL`. Kamau's previously-active `OUTGOING_VOICE_BARRED` is cleared (it's now redundant — the whole service is suspended). Returns `clearedRestrictions=["OUTGOING_VOICE_BARRED"]`.

02:00:06 — COMMITTING_FINAL_STATE: `sub-lm-put-master-fields` writes Kamau's `last_status_changed_at=NOW`; inserts pause-history row:

```
{
  "pause_id": "pause_01HZ_KAMAU_SUSPNP",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "originIntent": "SUSPEND_NP",
  "systemManaged": true,
  "dunningReasonCode": "DUNNING_ESCALATION_T3",
  "dunningCycleReference": "bil04-dunning-wave-2026-06-15-L3",
  "dunningEscalationLevel": 3,
  "outstandingDebtAmount": 3500.00,
  "outstandingDebtCurrency": "KES",
  "suspendedAt": "2026-06-15T02:00:06+03:00",
  "resumedAt": null
}
```

`sub-lm-commit-state` flips status PENDING_SUSPEND_NP → SUSPENDED.

02:00:07 — EMITTING_EVENT: `kafka-emit` writes full `SubscriptionSuspendedForNonPayment` payload (parent §6) to outbox. Outbox dispatcher publishes within seconds.

02:00:07.5: `notification-send` invokes NOT-01 with `notificationEventKind=SUBSCRIPTION_SUSPENDED_NON_PAYMENT` and event context `{dunningReasonCode: "DUNNING_ESCALATION_T3", outstandingDebtAmount: 3500.00, outstandingDebtCurrency: "KES", debtAmountTier: "STANDARD", suspendedAt: "2026-06-15T02:00:06+03:00"}` (KE config `customer_notification_enabled=true`). NOT-01 resolves Kamau's preferred channel + template + language, then sends the suspension notification.

02:00:08 — COMPLETED: `subscription-operation-finalize` sets `final_state=COMPLETED`. Total operation duration: ~8 seconds.

02:00:08+ (downstream consumers):

- BIL-04 consumes `SubscriptionSuspendedForNonPayment`, acknowledges its L3 step. The dunning state machine now waits for payment-received signal (which will trigger SUB-WF-RESUME-01 DUNNING_RESUME) or eventual L4 (TERMINATE).
- CVM analytics records the churn-risk event; debt tier STANDARD.
- EM-04 reporting indexes the suspension for monthly churn/debt metrics.

Kamau's full service is now suspended. ONT cannot authenticate, voice profile disabled. His `active_restrictions[]` array is empty (the L2 voice-bar was cleared since it's redundant). The pause-history row's `SUSPEND_NP` + `systemManaged=true` flags protect against voluntary resume via SUB-WF-PAUSE-01; only `DUNNING_RESUME` (BIL-04 after payment) or `ADMIN_FORCE_RESUME` will reactivate him.

Later — payment received, SUB-WF-RESUME-01 fires

When Kamau pays INV_01HZ via M-Pesa on (say) 2026-06-22, BIL-04 fires SUB-WF-RESUME-01 with `trigger=DUNNING_RESUME` referencing this suspension's `pause_id=pause_01HZ_KAMAU_SUSPNP`. SUB-WF-RESUME-01 reads the pause-history row's `originIntent=SUSPEND_NP`, confirms the resume trigger is allowed, flips subscription back to ACTIVE, closes the pause-history row. (Owned by SUB-WF-RESUME-01 DD, pending.)

5. Cross-DD coordination

Module	Direction	Contract
SUB-WF-SUSPEND-NP-01 (parent)	This satellite implements parent's contract	All triggers + state transitions defined in parent
SUB-WF-FRAMEWORK-01 (framework parent)	This satellite composes framework primitives	`subscription_operation` row written; BPMN started
SUB-WF-FRAMEWORK-01-WORKERS	This satellite references workers by topic	12 workers composed
SUB-LM-01	Workers write status + master fields + pause-history + clear restrictions	All SUB-LM-01 contracts honored
BIL-01	`bil01-emit-intent` worker	`SUSPEND_NP_INTENT`
BIL-04 Dunning	Calls trigger endpoint + consumes events	Drives at L3 escalation
FUL-04	`ful-call-restrict` worker with `intent=SUSPEND_SERVICE`	Network-level suspension
NOT-01	`notification-send` worker	Customer notification per operator config
SUB-WF-RESUME-01	Sibling DD (pending) — reads pause-history `originIntent=SUSPEND_NP`	Resume gating
CVM / EM-04	Consume Kafka events	Churn/debt analytics + reporting

5.1 Stub debt + open coordination items

- **WUG / WTZ / YASSN FLOW satellites** — pending operator deep dives.
- **SUB-WF-RESUME-01 DD** — pending; this satellite's pause-history-row flags must be honored by RESUME's role/trigger gating.

B — Current TZ

Status: To be filled in after codebase cartography review.

Legacy dunning suspension flow exists in TZ codebase but specific implementation shape (whether unified or per-trigger endpoints, exact AAA-disable sequencing, pause-history structure) requires source-code review. Until that cartography lands, this satellite does not assert specific legacy structural claims.